```python
from learning_module import LearningModule, Lesson, Quiz
from users.user import User
from users.educator import Educator
from users.learner import Learner
from users.parent import Parent

class Admin(User):
    """
    This is the class definition for the Admin class.
    """

    def __init__(self, email, first_name, last_name, username, password):
        """
        Constructor for the Admin class.
        """
        super().__init__(email, first_name, last_name, username, password,
        role="Admin")
        self.quiz = []


    # Admin related methods
    def dashboard(self, login_window=None):
        """
        This method is used to display the dashboard for the admin.

        :param login_window: The login window to be destroyed after successful login.
        """

        if login_window:
            login_window.destroy()
        while True:
            print(f"\nWelcome admin {self.first_name}\n")
            print("1. Adjust Challenges/Quizzes")
            print("2. Add/Remove Content")
            print("3. User Account Management")
            print("4. Logout")
            choice = int(input("Enter your choice: "))

            if choice == "1":
                self.
            elif choice == "2":
                self.
            elif choice == "3":
                print(f"User Account Management:\n")
                print("1. View Users")
                print("2. Add Users")
                print("3. Remove Users")
                option = int(input("Enter your option: "))
                if option == 1:
                    self.get_user()
                elif option == 2:
                    self.add_user()
                elif option == 3:
                    self.remove_user()
                else:
                    print("Please enter a valid option.")
            elif choice == "4":
                break
            else:
                print("Please choose a valid option.")

    def add_user(self, user):
        """
        This method is used to add a new user to the system.

        :param user: The user to be added to the system.
        """

        print("Enter user's details:")
        email = input("Enter email: ")
        first_name = input("Enter first name: ")
        last_name = input("Enter last name: ")
        dob = input("Enter date of birth (DD/MM/YYYY): ")
```

```
73          username = input("Enter username: ")
74          password = input("Enter password: ")
75          confirm_password = input("Confirm password: ")
76          role = input("\n1. Learner \n2. Educator \n3. Parent\nChoose your role: ")
77
78          if password != confirm_password:
79              print("Passwords do not match.")
80              return
81
82          if (
83              not email
84              or not first_name
85              or not last_name
86              or not dob
87              or not username
88              or not password
89              or not confirm_password
90              or not role
91          ):
92              print("Please fill in all the details.")
93              return
94
95          if username in [user.get_username() for user in self.users]:
96              print("Username already exists.")
97              return
98
99          # Creating a new user object depending on the role, and adding it to the
            users list
100         if role == "1":
101             user = Learner(email, first_name, last_name, username, password, dob)
102         elif role == "2":
103             user = Educator(email, first_name, last_name, username, password, dob)
104         elif role == "3":
105             user = Parent(email, first_name, last_name, username, password, dob)
106         else:
107             print("Please choose a valid role.")
108             return
109
110         # Saving the user details to the users.txt file
111         with open("./data/users.txt", "a") as file:
112             user_string = (
113                 f"{first_name},{last_name},{dob},{username},{password},{email}\n"
114             )
115             file.write(user_string)
116
117         self.users.append(user)
118         print(f"User {first_name} {last_name} added successfully!")
119
120     def remove_user(self, user):
121         try:
122             with open("./data/users.txt",'r') as file:
123             lines = file.readlines
124
125         # Initialize an empty list to store the modified content
126             new_lines = []
127
128         # Flag to check if the user was found and removed
129             user_removed = False
130
131         # Iterate through each line in the file
132             for line in lines:
133                 # Split the line into parts (assuming a specific format, like CSV)
134                 parts = line.strip().split(',')
135
136                 # Assuming the username is in the first part
137                 username = parts[0]
138
139                 # Check if the username matches the one to remove
140                 if username == username_to_remove:
141                     user_removed = True
142                 else:
143                     # If the username doesn't match, add the line to the new content
144                     new_lines.append(line)
```

```
145
146             # If the user was removed, write the updated content back to the file
147             if user_removed:
148                 with open(filename, 'w') as file:
149                     file.writelines(new_lines)
150                 print(f"User '{username_to_remove}' has been removed.")
151             else:
152                 print(f"User '{username_to_remove}' not found in the file.")
153
154         except FileNotFoundError:
155             print(f"File '{filename}' not found.")
156         except Exception as e:
157             print(f"An error occurred: {str(e)}")
158
159     def get_user(self, user):
160
161
162
163
164
165
166
167
168 # - Adjust challenges/quizzes
169 # - Add/remove content
170 # - User Account Management
171 #  - View users
172 #  - Add users
173 #  - Remove users (ban)
```